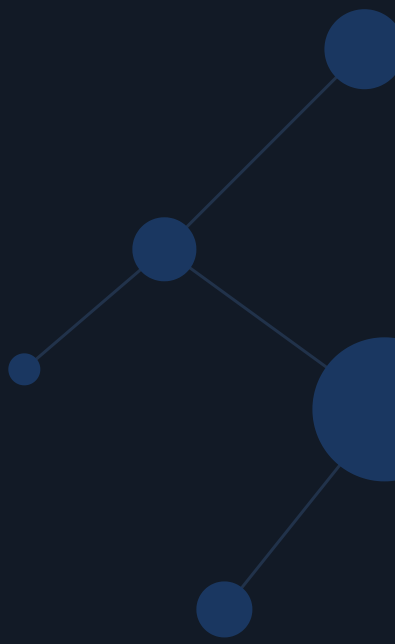
CSE4039 · ADVANCED TEXT & SOCIAL MEDIA ANALYTICS · PROJECT REPORT

PageRank on three networks

Web analytics with a from-scratch PageRank implementation, verified against NetworkX and applied to Zachary's Karate Club and a 100,000-page Wikipedia link graph, with a crawler for the VIT-AP website.



PROJECT LINKS

Presentation (slides)	https://github.com/swappy1409/atsma-project/tree/main/presentation
GitHub source code	https://github.com/swappy1409/atsma-project
Demo application	https://github.com/swappy1409/atsma-project/tree/main/demo-app
Demo video	https://github.com/swappy1409/atsma-project/tree/main/video
Report (this PDF)	https://github.com/swappy1409/atsma-project/tree/main/docs

TEAM
Swapnil (23BCB7031) · Shashikant (23BCB7077)

INSTITUTION
VIT-AP University

DATE
October 2026

Contents

1 Overview and objectives

2 Design and architecture

3 Input data

4 Intermediate data

5 Output data and results

6 Demo application

7 Presentation and demo video

8 How to run the project

9 Conclusions, limitations and references

A Appendix A: complete source code (pagerank_web_analytics.py)

B Appendix B: demo application core (JavaScript port) and requirements

1 Overview and objectives

PageRank (Brin & Page, 1998) ranks the nodes of a link network by the long-run share of time a *random surfer* spends on each node. With probability *d* (the damping factor, 0.85) the surfer follows a random out-link of the current page; with probability 1 – *d* it jumps to a page chosen uniformly at random. For page *i*:

$$PR(i) = (1 - d)/N + d \cdot \Sigma_{j \to i} PR(j) / L(j)$$

where *N* is the number of pages and *L(j)* the number of out-links of page *j*. Pages with no out-links (dangling nodes) would leak probability, so their rank is redistributed evenly over all pages. In matrix form PageRank is the principal eigenvector of the Google matrix $G = dM + (1 - d)/N \cdot 11^{T}$, found by power iteration $r \leftarrow G\ r$ until the L1 change between iterations is below 10^{-8}.

Objectives

- **Implement** PageRank from scratch: a readable dictionary version and a scalable SciPy sparse-matrix version.
- **Verify** both against NetworkX's `nx.pagerank` (max value difference, Spearman ρ, Kendall τ, top-10 overlap).
- **Apply** them to networks of very different size and nature: a 34-node social network and a 100K-page, 8.57M-link encyclopedia graph; include a polite crawler that builds the VIT-AP website graph.
- **Analyse** the damping factor, compare PageRank with other centrality measures, and explain what top-ranked pages have in common.

1.5×10^{-9}	1.0×10^{-7}	45 / 61	28.2%
max gap vs NetworkX, Karate	max gap, Wikipedia 20K sample	iterations, Karate / Wikipedia	of Wikipedia rank in the top 1% of pages

2 Design and architecture

The project is a single Jupyter notebook (`PageRank_Web_Analytics.ipynb`, mirrored as `src/pagerank_web_analytics.py`) organised as a five-stage pipeline. Every stage writes its results to `outputs/`, which feed the slides, this report and the browser demo.

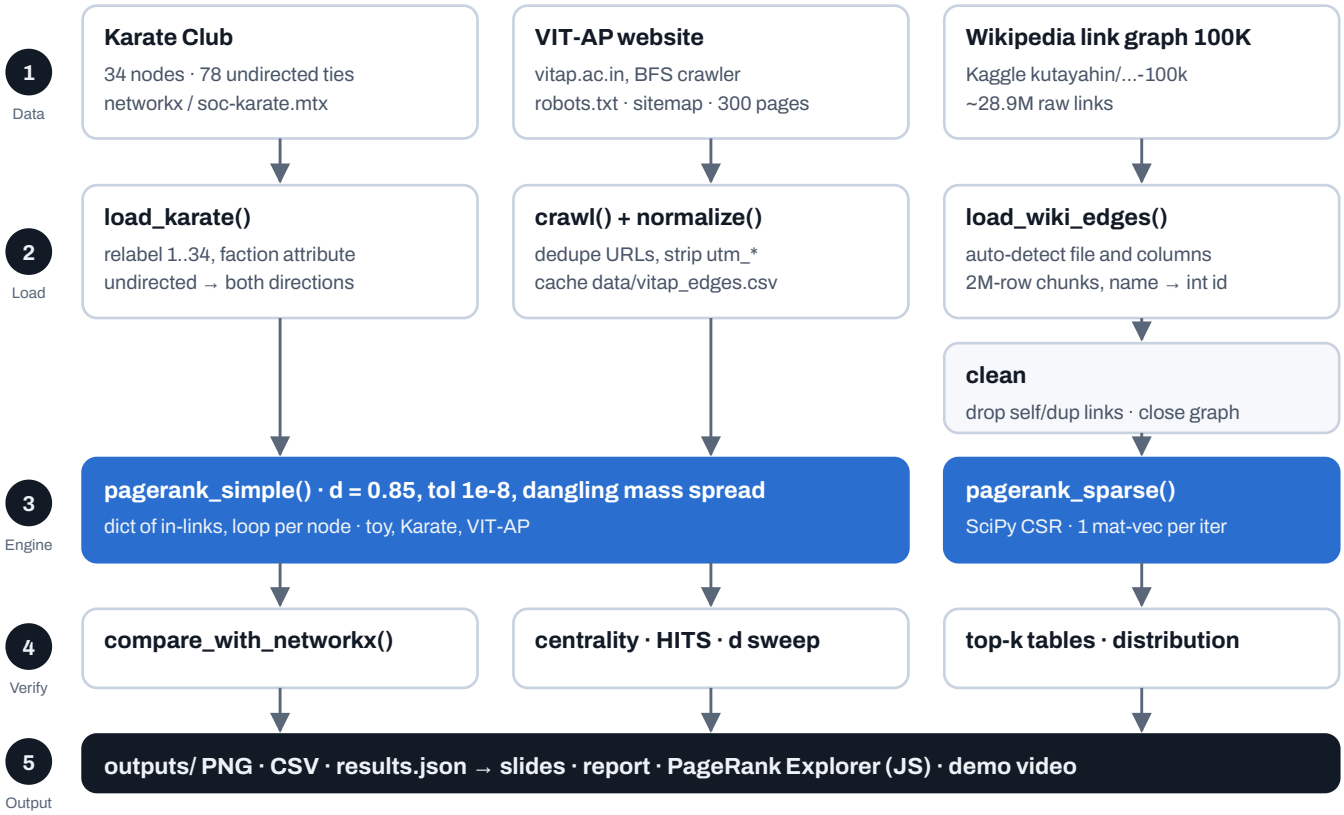


Figure 1. System architecture: data sources → loading and cleaning → PageRank engines → verification and analysis → outputs.

Key design decisions

- **Two engines, one algorithm.** `pagerank_simple` mirrors the formula for teaching and small graphs; `pagerank_sparse` builds the transposed transition matrix once (CSR) so each iteration is one sparse matrix–vector product. A dense 100K × 100K matrix would need ≈ 80 GB; the CSR matrix for 8.57M links needs ≈ 100 MB.
- **Dangling nodes** are handled explicitly in both engines by adding `d · Σ PR(dangling) / N` to every page, so total rank always sums to 1.
- **Convergence**: stop when the L1 change is below 10^{-8} (max 200 iterations). The L1-change history is returned for the convergence charts.
- **Robust Wikipedia loader**: inspects the downloaded files, picks the largest tabular file as the edge list, detects source/target columns by name patterns, reads it in 2M-row chunks and maps page names to `int32` ids so ~29M links fit in a few hundred MB.
- **Closed graph**: links whose target is not one of the source pages are dropped (automatic when targets outnumber sources by 1.5×), so PageRank is not dominated by pages outside the sample.
- **Verification at scale**: NetworkX cannot hold 8.57M edges in memory, so it is compared on a random induced subgraph of 20,000 pages.
- **Polite crawler** for vitap.ac.in: obeys robots.txt, seeds from sitemap.xml, normalises URLs (scheme, www, trailing slash, index pages, tracking parameters), stays on *.vitap.ac.in, skips non-HTML files, waits 0.5 s

between requests and caches the edge list.

Modules and functions

function	responsibility	used on
pagerank_simple(graph, d, tol, max_iter)	Dict-based power iteration with dangling-node handling; returns ranks and L1-change history	Toy, Karate, VIT-AP
pagerank_sparse(src, dst, n, d, tol, ...)	Same algorithm on a SciPy CSR transition matrix; one sparse mat-vec per iteration	Wikipedia 100K
compare_with_networkx(ours, nx_pr, k)	Max |diff|, Spearman ρ, Kendall τ and top-k overlap against nx.pagerank	All
top_table(pr, k, label, extra)	Top-k ranking table with optional labels and extra columns	All
plot_convergence(histories, fname)	Log-scale L1 change per iteration chart	All
load_karate()	Loads soc-karate .mtx if present, else NetworkX copy relabelled 1..34 with faction labels	Karate
normalize(url), in_scope(url), crawl(...)	Polite BFS crawler: robots.txt, sitemap seeding, URL normalisation, 0.5 s delay, 300-page cap	VIT-AP
get_wiki_dir(), detect_sep(), sniff(), pick_columns(), load_wiki_edges()	Locate/unzip dataset, detect format and source/target columns, chunked read, map names → int ids	Wikipedia

Technology stack

Python 3 · NumPy · SciPy (sparse CSR) · pandas · NetworkX (verification, centralities, HITS) · Matplotlib · requests + BeautifulSoup (crawler) · kagglehub (dataset download) · Jupyter. Demo app: HTML5 canvas + vanilla JavaScript, no server.

Repository structure

```
atsma-project/
├── README.md                        project overview and run instructions
├── PageRank_Web_Analytics.ipynb     main notebook (theory, code, all 3 networks, charts)
├── requirements.txt
├── src/
│   └── pagerank_web_analytics.py    same code as a plain script (# %% cells)
├── data/
│   ├── README.md                    how to obtain the Wikipedia dataset
│   └── input/                       karate_edges.csv, toy_links.csv, example_site_edges.csv
├── intermediate/                    transition matrices, iteration traces, adjacency list, ...
├── outputs/                         charts (PNG), top-k tables (CSV), results.json, summary.csv
├── demo-app/
│   └── index.html                   PageRank Explorer (stand-alone, open in any browser)
├── video/                           PageRank_Demo_Video.mp4, .srt subtitles, narration script
└── docs/
    └── PageRank_Project_Report.pdf  this report
```

3 Input data

dataset	type	nodes	links	source	status
A · Toy web	directed	4	5	Hand-made example (same style as the Kaggle reference notebook)	Used
B · Zachary's Karate Club	undirected social	34	78	Zachary (1977); networkx.karate_club_graph(), identical to soc-karate (networkrepository.com)	Used
C · Wikipedia Link Graph 100K	directed hyperlinks	99,998	8,572,670 (after cleaning; ~28.9M raw)	Kaggle: kutayahin/wikipedia-link-graph-100k	Used
D · VIT-AP website	directed hyperlinks	≤ 300 pages	–	Own crawl of https://vitap.ac.in/	Crawler implemented; not run for this submission

3.1 Toy web (data/input/toy_links.csv)

```
source,target
A,B
A,C
B,C
C,A
D,C
```

3.2 Karate Club edge list (data/input/karate_edges.csv, all 78 ties)

Nodes use the 1-based labels of the original paper. Node 1 is the instructor (“Mr. Hi”), node 34 the administrator (“Officer”). Each tie counts as a link in both directions.

1 – 2	1 – 20	3 – 9	6 – 17	20 – 34	26 – 32
1 – 3	1 – 22	3 – 10	7 – 17	21 – 33	27 – 30
1 – 4	1 – 32	3 – 14	9 – 31	21 – 34	27 – 34
1 – 5	2 – 3	3 – 28	9 – 33	23 – 33	28 – 34
1 – 6	2 – 4	3 – 29	9 – 34	23 – 34	29 – 32
1 – 7	2 – 8	3 – 33	10 – 34	24 – 26	29 – 34
1 – 8	2 – 14	4 – 8	14 – 34	24 – 28	30 – 33
1 – 9	2 – 18	4 – 13	15 – 33	24 – 30	30 – 34
1 – 11	2 – 20	4 – 14	15 – 34	24 – 33	31 – 33
1 – 12	2 – 22	5 – 7	16 – 33	24 – 34	31 – 34
1 – 13	2 – 31	5 – 11	16 – 34	25 – 26	32 – 33
1 – 14	3 – 4	6 – 7	19 – 33	25 – 28	32 – 34
1 – 18	3 – 8	6 – 11	19 – 34	25 – 32	33 – 34

3.3 Wikipedia Link Graph 100K

Downloaded from Kaggle (`kutayahin/wikipedia-link-graph-100k`) into `data/wikipedia/`, or automatically with `kagglehub`. The raw dataset (~28.9M links) is too large to ship in the repository; `data/README.md` explains how to get it. Column names and file format are detected automatically by `load_wiki_edges()`.

3.4 VIT-AP crawler configuration

```
START_URL  = "https://vitap.ac.in/"     ALLOWED   = "vitap.ac.in" (and subdomains)
MAX_PAGES  = 300                        DELAY     = 0.5 s between requests
TIMEOUT    = 15 s                       USER_AGENT = "...ATSMA-student-PageRank-project/1.0;
educational crawl"
SKIP_EXT   = pdf, images, media, archives, office files, css, js, xml, json
CACHE      = data/vitap_edges.csv (+ .titles.json)
```

> The VIT-AP crawl needs live internet access to vitap.ac.in and was not part of the runs whose results are reported here, so no VIT-AP outputs appear in sections 4–5. Running the notebook with internet access produces `vitap_top15.csv` and three VIT-AP charts automatically.

3.5 Example input for the demo app (data/input/example_site_edges.csv)

A made-up 12-page university website (not crawled data), pre-loaded in the demo's "Your own graph" view to illustrate the navigation effect.

```
# example: a small university      academics home                    news-convocation news
website (made-up pages)            academics cse-dept                news-hackathon news
home admissions                    academics ece-dept                news-hackathon cse-dept
home academics                     academics admissions              cse-dept home
home placements                    placements home                   cse-dept academics
home contact                       placements contact                ece-dept home
home news                          contact home                      ece-dept academics
admissions home                    news home                         fees admissions
admissions fees                    news news-convocation             fees contact
admissions contact                 news news-hackathon               cse-dept fees-pdf
```

4 Intermediate data

All tables below are produced by the project code and saved in `intermediate/`.

4.1 Toy web: transition matrix M and Google matrix G (d = 0.85)

M[i, j] = 1/L(j) if page j links to page i (column-stochastic). G = 0.85 M + 0.15/4.

M	A	B	C	D
A	0.0	0.0	1.0	0.0
B	0.5	0.0	0.0	0.0
C	0.5	1.0	0.0	1.0
D	0.0	0.0	0.0	0.0

G	A	B	C	D
A	0.0375	0.0375	0.8875	0.0375
B	0.4625	0.0375	0.0375	0.0375
C	0.4625	0.8875	0.0375	0.8875
D	0.0375	0.0375	0.0375	0.0375

4.2 Toy web: power-iteration trace (intermediate/toy_iteration_trace.csv)

iteration	A	B	C	D	l1_delta
0	0.2500	0.2500	0.2500	0.2500	–
1	0.2500	0.1437	0.5687	0.0375	6.37e-01
2	0.5209	0.1437	0.2978	0.0375	5.42e-01
3	0.2906	0.2589	0.4130	0.0375	4.61e-01
4	0.3885	0.1610	0.4130	0.0375	1.96e-01
5	0.3885	0.2026	0.3714	0.0375	8.32e-02
10	0.3751	0.1949	0.3925	0.0375	9.23e-03
20	0.3725	0.1958	0.3942	0.0375	5.68e-05
30	0.3725	0.1958	0.3941	0.0375	3.49e-07
40	0.3725	0.1958	0.3941	0.0375	2.15e-09
47	0.3725	0.1958	0.3941	0.0375	8.61e-11

Converged after 47 iterations at tolerance 10^{-10}. NetworkX gives A 0.3725, B 0.1958, C 0.3942, D 0.0375.

4.3 Karate Club: adjacency list (intermediate/karate_adjacency_list.csv)

node	faction	degree	neighbours
1	Mr. Hi	16	2 3 4 5 6 7 8 9 11 12 13 14 18 20 22 32
2	Mr. Hi	9	1 3 4 8 14 18 20 22 31
3	Mr. Hi	10	1 2 4 8 9 10 14 28 29 33
4	Mr. Hi	6	1 2 3 8 13 14
5	Mr. Hi	3	1 7 11
6	Mr. Hi	4	1 7 11 17
7	Mr. Hi	4	1 5 6 17
8	Mr. Hi	4	1 2 3 4
9	Mr. Hi	5	1 3 31 33 34
10	Officer	2	3 34
11	Mr. Hi	3	1 5 6
12	Mr. Hi	1	1
13	Mr. Hi	2	1 4

node	faction	degree	neighbours
14	Mr. Hi	5	1 2 3 4 34
15	Officer	2	33 34
16	Officer	2	33 34
17	Mr. Hi	2	6 7
18	Mr. Hi	2	1 2
19	Officer	2	33 34
20	Mr. Hi	3	1 2 34
21	Officer	2	33 34
22	Mr. Hi	2	1 2
23	Officer	2	33 34
24	Officer	5	26 28 30 33 34
25	Officer	3	26 28 32
26	Officer	3	24 25 32
27	Officer	2	30 34
28	Officer	4	3 24 25 34
29	Officer	3	3 32 34
30	Officer	4	24 27 33 34
31	Officer	4	2 9 33 34
32	Officer	6	1 25 26 29 33 34
33	Officer	12	3 9 15 16 19 21 23 24 30 31 32 34
34	Officer	17	9 10 14 15 16 19 20 21 23 24 27 28 29 30 31 32 33

4.4 Karate Club: PageRank after each iteration (selected nodes)

iter	L1 change	node 34	node 1	node 33	node 3	node 2	node 17
0	–	0.02941	0.02941	0.02941	0.02941	0.02941	0.02941
1	7.46e-01	0.14858	0.13427	0.09755	0.05834	0.06347	0.01691
2	4.24e-01	0.07256	0.07989	0.05538	0.05040	0.04129	0.01846
3	2.57e-01	0.11758	0.10688	0.08195	0.05801	0.05728	0.01783
5	9.26e-02	0.10687	0.10040	0.07563	0.05731	0.05442	0.01732
10	7.27e-03	0.10033	0.09683	0.07127	0.05701	0.05271	0.01683
15	6.16e-04	0.10094	0.09704	0.07171	0.05708	0.05289	0.01679
20	5.40e-05	0.10091	0.09700	0.07169	0.05708	0.05288	0.01679
30	7.78e-07	0.10092	0.09700	0.07169	0.05708	0.05288	0.01678
40	3.62e-08	0.10092	0.09700	0.07169	0.05708	0.05288	0.01678
45	7.91e-09	0.10092	0.09700	0.07169	0.05708	0.05288	0.01678

Every node starts at 1/34 = 0.02941. The full 34-node trace for all 45 iterations is in intermediate/karate_iteration_trace.csv.

4.5 Karate Club: convergence history (intermediate/karate_convergence_history.csv)

iteration	L1 change (dict)	L1 change (sparse)	ratio to previous
1	7.463e-01	7.463e-01	–
2	4.238e-01	4.238e-01	0.568
3	2.566e-01	2.566e-01	0.605
5	9.256e-02	9.256e-02	0.616

iteration	L1 change (dict)	L1 change (sparse)	ratio to previous
10	7.268e-03	7.268e-03	0.590
20	5.400e-05	5.400e-05	0.634
30	7.779e-07	7.779e-07	0.729
40	3.621e-08	3.621e-08	0.737
45	7.913e-09	7.913e-09	0.738

The error shrinks by a near-constant factor of about 0.6–0.75 per step (geometric mean 0.67), within the theoretical bound d = 0.85.

4.6 Karate Club: damping-factor sweep (intermediate/karate_damping_table.csv)

d	iterations	PR(34)	PR(1)	PR(33)	top 5 nodes	max ÷ min PR
0.5	19	0.08	0.0764	0.0588	34 1 33 3 2	4.68
0.65	25	0.0899	0.0862	0.0648	34 1 33 3 2	6.52
0.75	32	0.0957	0.0919	0.0684	34 1 33 3 2	8.2
0.85	45	0.1009	0.097	0.0717	34 1 33 3 2	10.55
0.95	70	0.1061	0.1012	0.075	34 1 33 3 2	14.19

4.7 Wikipedia: cleaning and graph construction

step / statistic	value
Raw links in the dataset	~28.9 million
Steps	drop NaN rows → remove self-links → remove duplicate (source, target) pairs → close graph (keep links whose target is also a source page) → re-index pages 0..N−1
Final graph: pages	99,998
Final graph: links	8,572,670 (mean out-degree 85.7)
Dangling pages (no out-links)	203
Closed graph	True
Iterations to converge (tol 10^{-8})	61
Verification subgraph	20,000 randomly chosen pages (seed 0), induced links only

5 Output data and results

5.1 Summary across networks (outputs/summary.csv)

network	type	nodes	edges	iterations	max |ours – NetworkX|
Karate Club	undirected social	34	78	45	1.49e-09
Wikipedia 100K	directed encyclopedia	99,998	8,572,670	61	1.03e-07

5.2 Verification against NetworkX

comparison	max |diff|	Spearman ρ	Kendall τ	top-10 overlap
Karate Club (full graph)	1.49e-09	1.0000	1.0000	100%
Wikipedia (20,000-page random induced subgraph)	1.03e-07	1.0000	1.0000	100%

5.3 Karate Club: full ranking, all 34 members (intermediate/karate_all_nodes_pagerank.csv)

rank	node	faction	deg	PR (ours)	PR (NetworkX)	|diff|	betweenness	eigenvector	closeness
1	34	Officer	17	0.100919	0.100919	1.4e-09	0.3041	0.3734	0.55
2	1	Mr. Hi	16	0.096997	0.096997	1.5e-09	0.4376	0.3555	0.569
3	33	Officer	12	0.071693	0.071693	1.1e-09	0.1452	0.3087	0.5156
4	3	Mr. Hi	10	0.057079	0.057079	3.5e-11	0.1437	0.3172	0.5593
5	2	Mr. Hi	9	0.052877	0.052877	4.3e-10	0.0539	0.266	0.4853
6	32	Officer	6	0.037158	0.037158	4.0e-10	0.1383	0.191	0.541
7	4	Mr. Hi	6	0.035860	0.035860	3.5e-10	0.0119	0.2112	0.4648
8	24	Officer	5	0.031523	0.031523	5.5e-10	0.0176	0.1501	0.3929
9	9	Mr. Hi	5	0.029766	0.029766	1.5e-10	0.0559	0.2274	0.5156
10	14	Mr. Hi	5	0.029536	0.029536	1.3e-10	0.0459	0.2265	0.5156
11	7	Mr. Hi	4	0.029111	0.029111	8.6e-10	0.03	0.0795	0.3837
12	6	Mr. Hi	4	0.029111	0.029111	8.6e-10	0.03	0.0795	0.3837
13	30	Officer	4	0.026289	0.026289	4.6e-10	0.0029	0.135	0.3837
14	28	Officer	4	0.025640	0.025640	3.4e-10	0.0223	0.1335	0.4583
15	31	Officer	4	0.024590	0.024590	1.8e-10	0.0144	0.1748	0.4583
16	8	Mr. Hi	4	0.024490	0.024490	2.2e-10	0.0	0.171	0.44
17	11	Mr. Hi	3	0.021978	0.021978	5.8e-10	0.0006	0.076	0.3793
18	5	Mr. Hi	3	0.021978	0.021978	5.8e-10	0.0006	0.076	0.3793
19	25	Officer	3	0.021076	0.021076	2.9e-10	0.0022	0.0571	0.375
20	26	Officer	3	0.021006	0.021006	3.1e-10	0.0038	0.0592	0.375
21	20	Mr. Hi	3	0.019605	0.019605	6.4e-11	0.0325	0.1479	0.5
22	29	Officer	3	0.019573	0.019573	1.7e-10	0.0018	0.1311	0.4521
23	17	Mr. Hi	2	0.016784	0.016784	5.0e-10	0.0	0.0236	0.2845
24	27	Officer	2	0.015044	0.015044	2.3e-10	0.0	0.0756	0.3626
25	13	Mr. Hi	2	0.014645	0.014645	1.7e-10	0.0	0.0843	0.3708
26	18	Mr. Hi	2	0.014559	0.014559	1.6e-10	0.0	0.0924	0.375
27	22	Mr. Hi	2	0.014559	0.014559	1.6e-10	0.0	0.0924	0.375
28	23	Officer	2	0.014536	0.014536	2.0e-10	0.0	0.1014	0.3708
29	16	Officer	2	0.014536	0.014536	2.0e-10	0.0	0.1014	0.3708

rank	node	faction	deg	PR (ours)	PR (NetworkX)	|diff|	betweenness	eigenvector	closeness
30	15	Officer	2	0.014536	0.014536	2.0e-10	0.0	0.1014	0.3708
31	19	Officer	2	0.014536	0.014536	2.0e-10	0.0	0.1014	0.3708
32	21	Officer	2	0.014536	0.014536	2.0e-10	0.0	0.1014	0.3708
33	10	Officer	2	0.014309	0.014309	1.0e-10	0.0008	0.1027	0.4342
34	12	Mr. Hi	1	0.009565	0.009565	1.1e-10	0.0	0.0529	0.3667

5.4 Karate Club: Spearman rank correlation between centralities (outputs/karate_centrality_correlation.csv)

	PageRank	degree	betweenness	eigenvector	closeness
PageRank	1.0	0.979	0.879	0.676	0.825
degree	0.979	1.0	0.905	0.777	0.895
betweenness	0.879	0.905	1.0	0.694	0.898
eigenvector	0.676	0.777	0.694	1.0	0.856
closeness	0.825	0.895	0.898	0.856	1.0

Nodes **34** (administrator) and **1** (instructor), the two leaders around whom the club split, have by far the highest PageRank and together hold 19.8% of all rank. PageRank correlates strongly with degree ($\rho = 0.979$) because the graph is undirected, but it still separates members with equal degree by the importance of their friends: members 24, 9 and 14 all have five friends yet score 0.0315, 0.0298 and 0.0295.

5.5 Wikipedia 100K: top 20 articles (outputs/wikipedia_top20.csv)

rank	page	pagerank	in_links	out_links
1	ISBN (identifier)	0.009806	51,534	185
2	Doi (identifier)	0.006266	31,461	152
3	ISSN (identifier)	0.005016	23,311	139
4	Wayback Machine	0.004716	24,295	67
5	S2CID (identifier)	0.003286	17,819	34
6	OCLC (identifier)	0.002311	11,887	26
7	PMID (identifier)	0.002258	9,911	36
8	Bibcode (identifier)	0.002046	12,396	7
9	JSTOR (identifier)	0.001815	11,253	26
10	United States	0.001673	9,016	381
11	The New York Times	0.001621	8,003	227
12	PMC (identifier)	0.001471	6,368	39
13	Time zone	0.001293	9,293	331
14	Hdl (identifier)	0.001110	5,974	14
15	World War II	0.001091	6,186	389
16	International Organization for Standardization	0.001040	428	150
17	United Kingdom	0.001037	6,053	564
18	The Guardian	0.000953	5,298	174
19	Daylight saving time	0.000936	6,899	118
20	France	0.000888	6,089	545

Citation identifier pages (ISBN, DOI, ISSN, S2CID, OCLC, PMID, ...) dominate because almost every referenced article links to them. Rank is highly concentrated: the top 1% of pages hold **28.2%** of all PageRank, and the rank–score curve is a heavy tail. PageRank differs from counting in-links: **International Organization for Standardization** has only 428 in-links (14× fewer than France) yet ranks 16th, because its in-links come from the top identifier pages.

5.6 Charts (outputs/*.png)

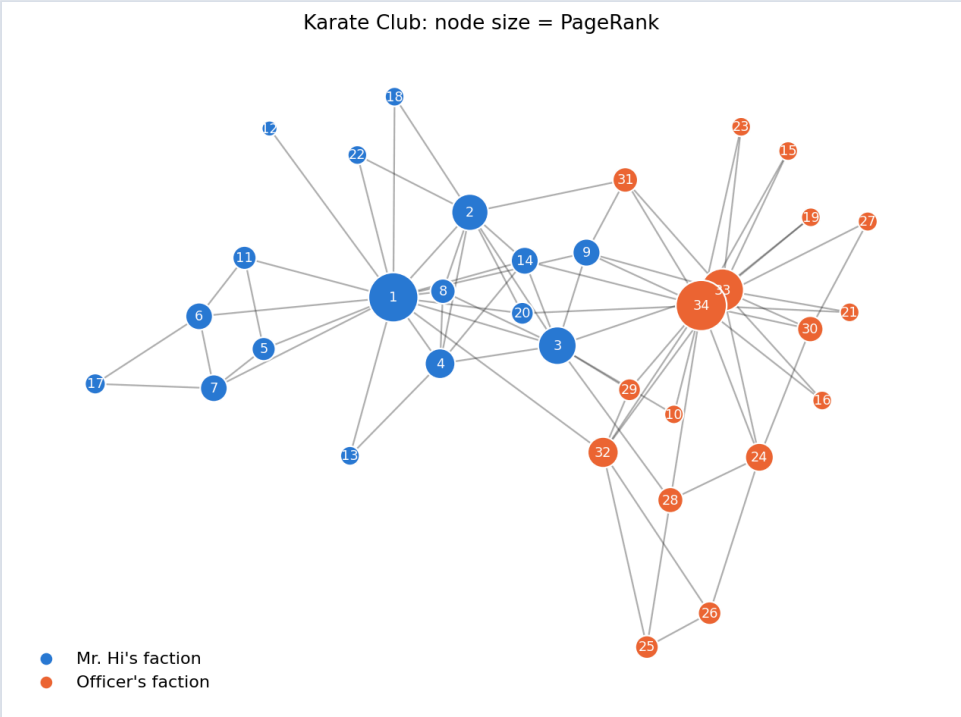


Figure 4. Karate Club network; node size = PageRank, colour = faction after the split.

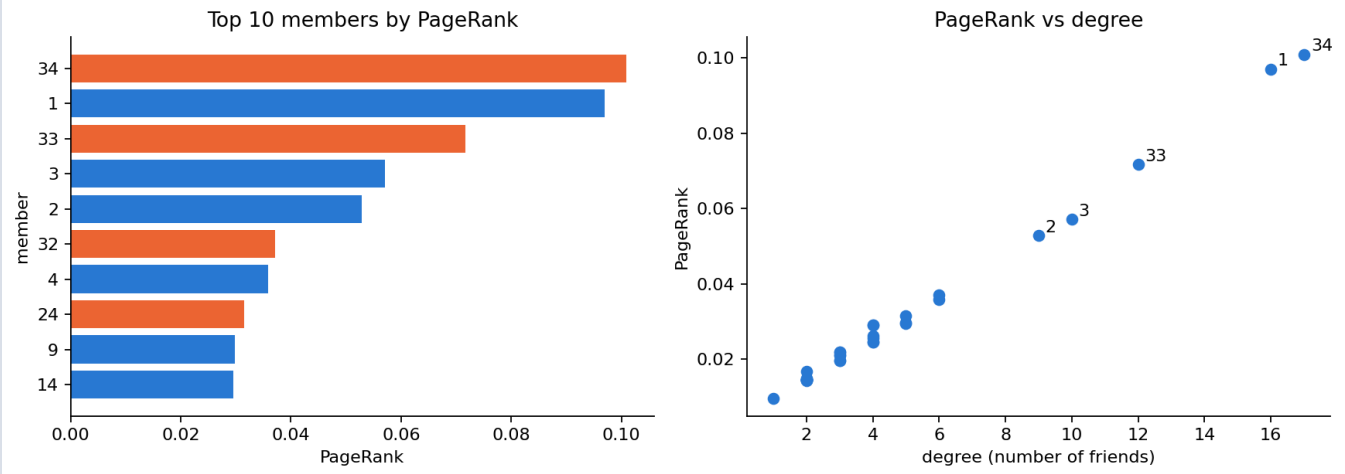


Figure 5. Top-10 members by PageRank (left) and PageRank against degree (right).

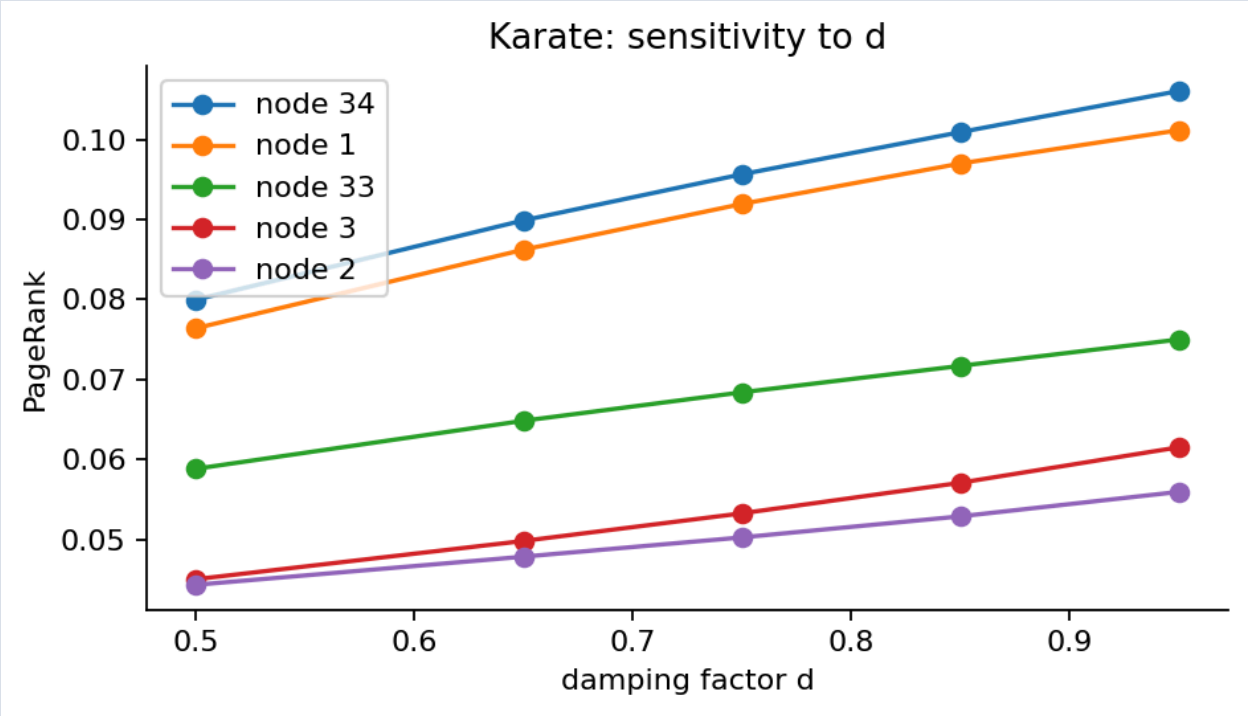


Figure 6. PageRank of the five top members as the damping factor d varies.

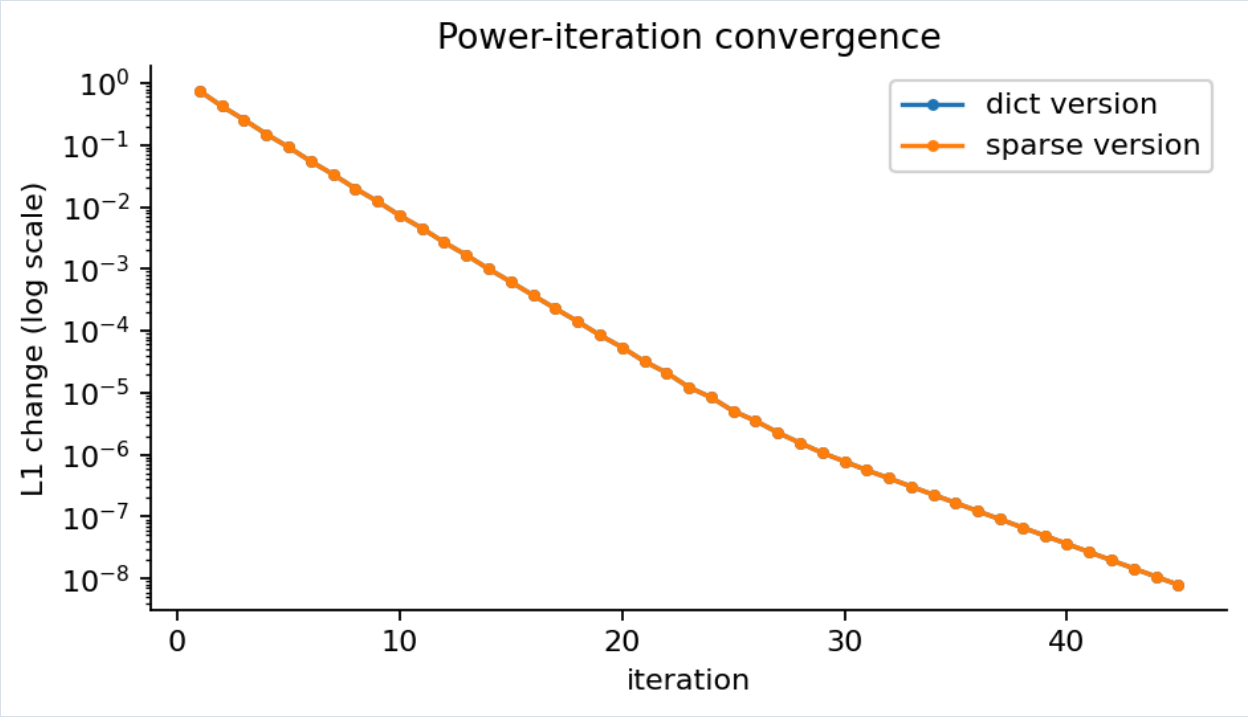


Figure 7. Karate Club power-iteration convergence; dict and sparse versions overlap.

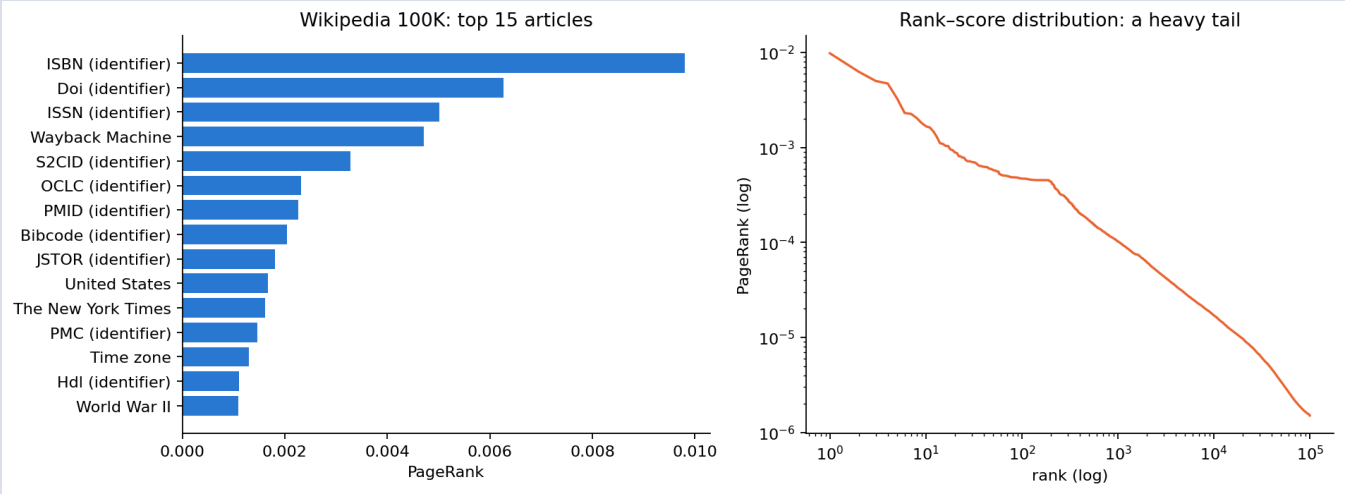


Figure 8. Wikipedia 100K: top 15 articles (left) and rank–score distribution on log–log axes (right).

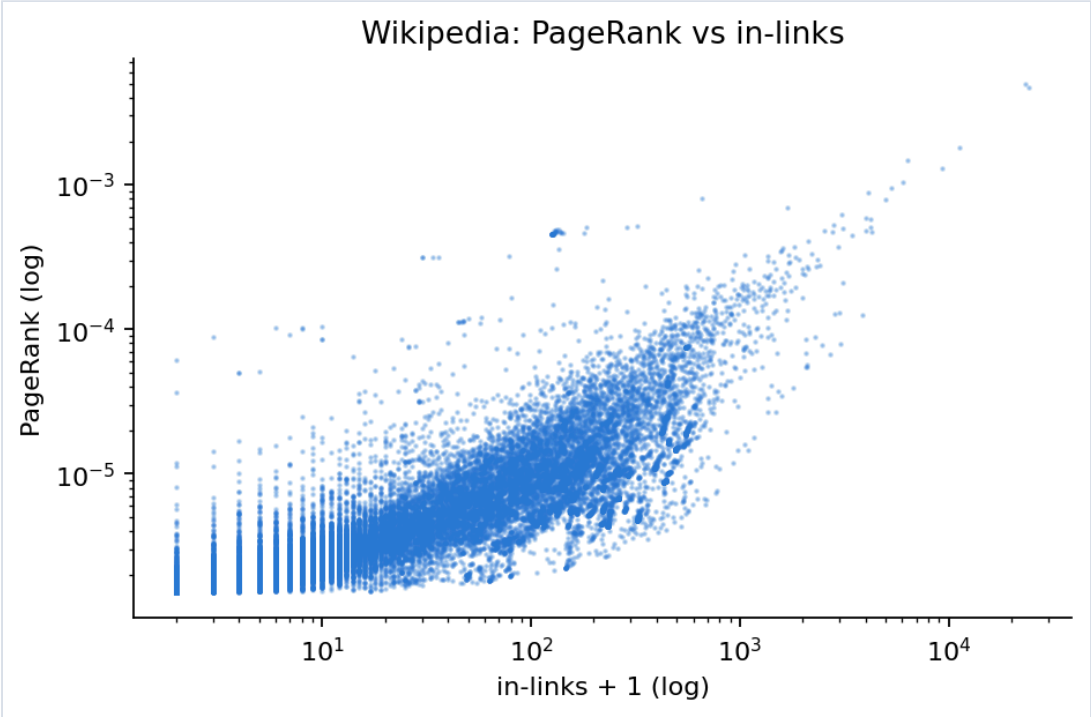


Figure 9. Wikipedia: PageRank against number of in-links (30,000-page sample, log–log).

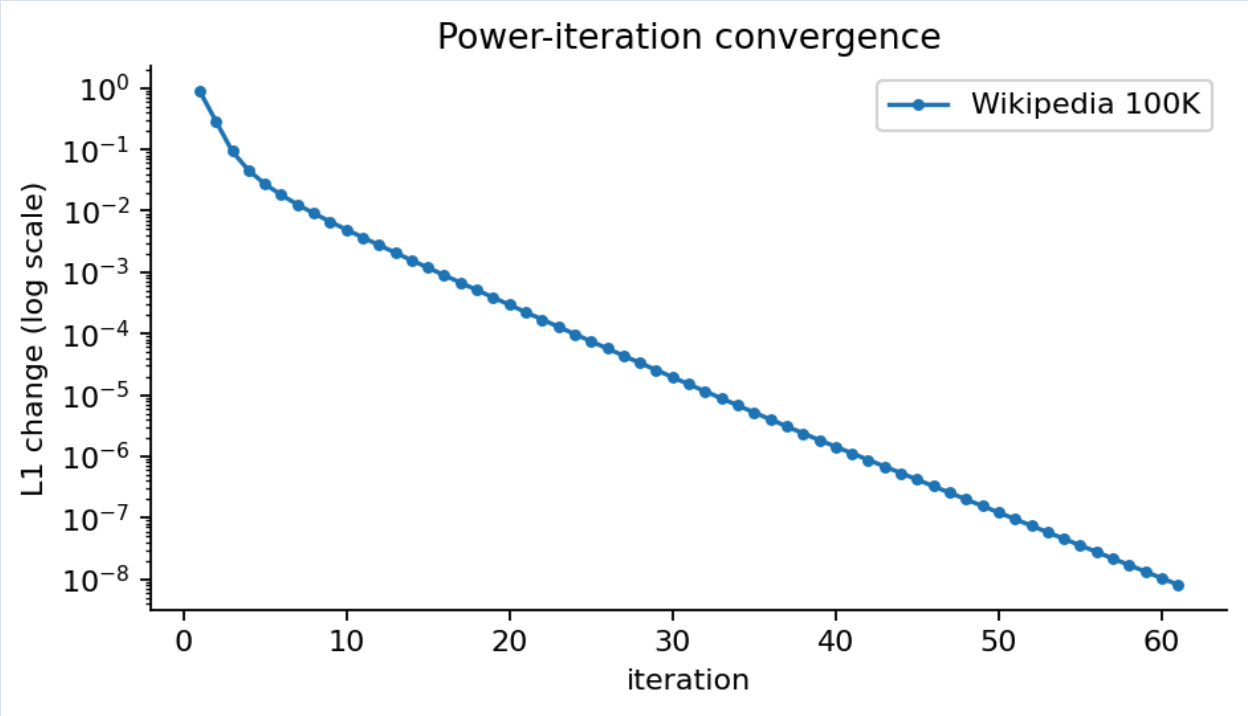


Figure 10. Wikipedia 100K power-iteration convergence (61 iterations).

6 Demo application: PageRank Explorer

Link: https://github.com/swappy1409/atsma-project/tree/main/demo-app · stand-alone copy: `demo-app/index.html` in the repository (download and open in any browser).

The app ports `pagerank_simple()` to JavaScript and runs it live in the browser. Its Karate Club numbers match the Python output to four decimals. Four views:

- **Toy web**: step through power iteration one click at a time or run to convergence; node size and the table update every step.
- **Karate Club**: animated iterations, a damping slider (0.05–0.99) that recomputes ranks, iteration count and max/min ratio live, and click-to-select members.
- **Wikipedia 100K**: top-20 PageRank vs in-link chart and graph statistics from the offline sparse run.
- **Your own graph**: paste any edge list (directed or undirected), choose d, and get a ranking table and a force-directed drawing.

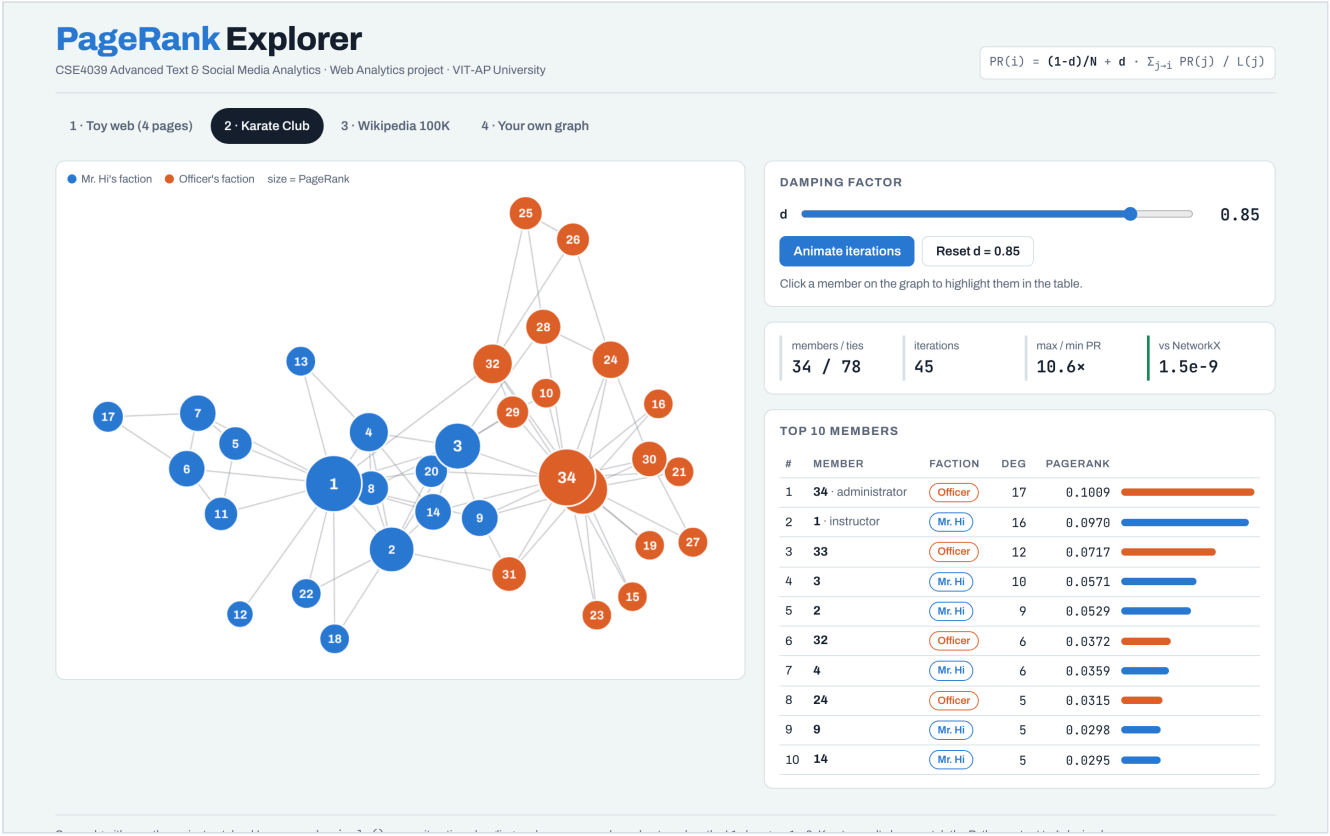


Figure 11. Karate Club view with the damping slider and live top-10 table.

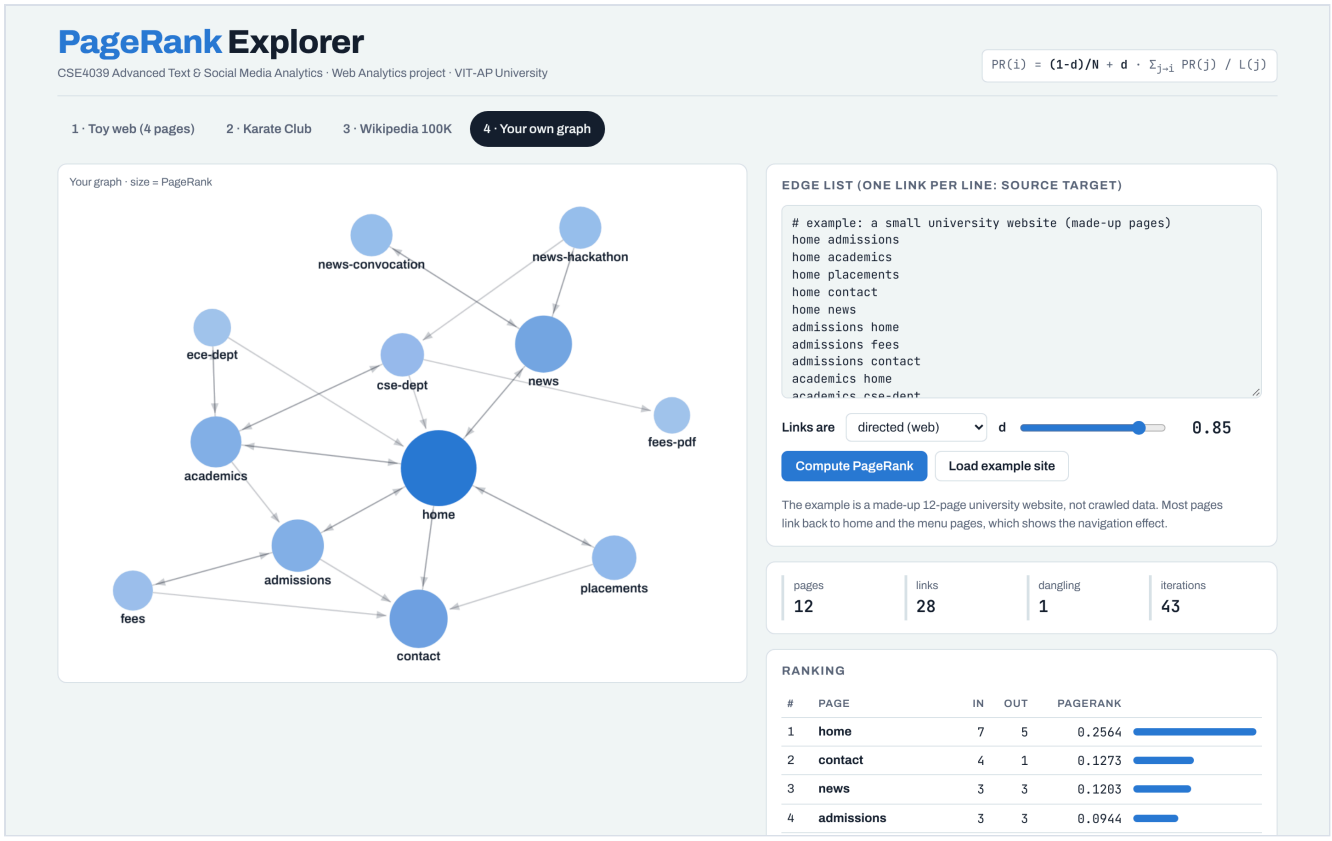


Figure 12. “Your own graph” view ranking the example university website; home ranks first (navigation effect).

7 Presentation and demo video

Presentation

Link: https://github.com/swappy1409/atsma-project/tree/main/presentation — 17 slides with speaker notes: objective, the algorithm, worked example, architecture, the two engines, datasets, Karate Club results, centrality comparison, damping, verification, Wikipedia results, PageRank vs in-links, convergence, demo, conclusions and links. The slides are in the repository as `presentation/PageRank_Presentation.pdf`.

Demo video

Link: https://github.com/swappy1409/atsma-project/tree/main/video · file: `video/PageRank_Demo_Video.mp4` (4 min 4 s, 1920×1080, H.264). Narration is provided as burned-in subtitles with on-screen annotations (highlight boxes, step labels and a visible cursor); the same text is in `video/PageRank_Demo_Video.srt` for YouTube captions or for recording a voice-over.

time	chapter
0:00	Title
0:08	Source code walkthrough: pagerank_simple() and pagerank_sparse()
0:56	Demo app: 4-page toy web, step-by-step power iteration
1:39	Karate Club: animated iterations, top members, NetworkX check, damping slider, node selection
2:38	Wikipedia 100K results and the ISO in-link example
3:10	Your own graph: edge-list input, directed vs undirected
3:36	Notebook output charts
3:55	Links

8 How to run the project

```
git clone https://github.com/swappy1409/atsma-project
cd atsma-project
pip install -r requirements.txt

# Wikipedia data (one of):
#   a) download the zip from https://www.kaggle.com/datasets/kutayahin/wikipedia-link-graph-100k
#      and put it (zipped or unzipped) in data/wikipedia/
#   b) let the notebook download it with kagglehub (needs a Kaggle API token; works on Kaggle/Colab)

jupyter notebook PageRank_Web_Analytics.ipynb      # Run All, about 10 minutes
# or:  python src/pagerank_web_analytics.py

# Karate Club needs no download. The VIT-AP crawl (3-5 min) needs internet access and is cached.
# Wikipedia (~28.9M raw links) needs 2-4 GB RAM; on a small laptop set MAX_EDGES = 5_000_000.
# Results appear in outputs/. Demo app: open demo-app/index.html in a browser.
```

9 Conclusions, limitations and references

Conclusions

1. **Correctness.** The from-scratch power iteration reproduces NetworkX (max difference 1.5×10^{-9} on Karate Club, 1.0×10^{-7} on the Wikipedia sample; identical top 10) and converges in 45 and 61

iterations.

2. **Karate Club.** PageRank identifies the two leaders (34 and 1) who split the club without any knowledge of the conflict. On an undirected graph it behaves much like degree, yet still rewards connections to important members.
3. **Wikipedia.** With sparse matrices PageRank handles 8.57M links easily. Scores are heavy-tailed and dominated by hub pages; link quality matters more than link count.
4. **Damping factor.** Higher d makes ranks more unequal (max/min 4.7× at d = 0.5, 14.2× at d = 0.95) but slows convergence (19 → 70 iterations); the top five never change.

Limitations

- The VIT-AP crawl was not run for this submission; it is capped at 300 pages and ignores JavaScript-rendered links.
- The Wikipedia graph is a closed 100K-page sample, so links to pages outside it are excluded.
- Karate Club is tiny and undirected, which makes PageRank close to degree centrality.

References

- Brin, S. & Page, L. (1998). The anatomy of a large-scale hypertextual web search engine. *Computer Networks and ISDN Systems*, 30, 107–117.
- Zachary, W. W. (1977). An information flow model for conflict and fission in small groups. *Journal of Anthropological Research*, 33(4), 452–473.
- Rossi, R. A. & Ahmed, N. K. (2015). The Network Data Repository with interactive graph analytics and visualization. AAAI. networkrepository.com
- Kaggle dataset: kutayahin/wikipedia-link-graph-100k. Kaggle reference notebook: vinitkp/page-rank-algorithm.
- Hagberg, A., Schult, D. & Swart, P. (2008). Exploring network structure, dynamics, and function using NetworkX. SciPy 2008.

Appendix A Complete source code: src/pagerank_web_analytics.py

Identical to the code cells of `PageRank_Web_Analytics.ipynb` (markdown cells appear as `# %% [markdown]` comments). 801 lines.

```
# %% [markdown]
# # Web Analytics: PageRank on Three Networks
# **Course:** Advanced Text and Social Media Analytics
#
# We implement the **PageRank** algorithm from scratch (power iteration), verify it against
# NetworkX's `nx.pagerank`, and apply it to three networks of very different size and nature:
#
# | # | Network | Type | Size |
# |---|---|---|---|
# | A | **VIT-AP website** (`vitap.ac.in`), crawled by us | directed, web pages → hyperlinks | a few hundred pages |
# | B | **Wikipedia Link Graph - 100K pages** (Kaggle: `kutayahin/wikipedia-link-graph-100k`) | directed, articles → links | ~100K pages, ~28.9M links |
# | C | **Zachary's Karate Club** (`soc-karate`, networkrepository.com) | undirected social network | 34 members, 78 ties |
#
# **Notebook outline**
# 1. The PageRank idea and a tiny worked example
# 2. Two from-scratch implementations (readable dict version and a fast sparse-matrix version)
# 3. Dataset C: Karate Club (small, easy to visualise, ground truth available)
# 4. Dataset A: VIT-AP website crawl
# 5. Dataset B: Wikipedia 100K link graph
# 6. Comparison and conclusions
#
# Every figure and table is also saved into `outputs/` so it can go straight into the slides.

# %% [markdown]
# ## 0. Setup
# Run once if a package is missing (Colab already has most of these):
# ```
# pip install networkx scipy pandas matplotlib requests beautifulsoup4 kagglehub
# ```

# %%
import os, re, io, json, time, math, glob, random, collections
from urllib.parse import urljoin, urlparse, urldefrag
import urllib.robotparser

import numpy as np
import pandas as pd
import scipy.sparse as sp
import networkx as nx
import matplotlib.pyplot as plt
from scipy.stats import spearmanr, kendalltau
try:
    from IPython.display import display
except ImportError:
    display = print

plt.rcParams.update({"figure.dpi": 110, "axes.spines.top": False, "axes.spines.right": False})
OUT = "outputs"
os.makedirs(OUT, exist_ok=True)
RESULTS = {}          # collected numbers for the slides (saved to outputs/results.json)
random.seed(42); np.random.seed(42)

def save_fig(name):
    plt.tight_layout()
    plt.savefig(os.path.join(OUT, name), dpi=160, bbox_inches="tight")
    plt.show()

# %% [markdown]
# ## 1. What PageRank computes
#
# A "random surfer" clicks a random outgoing link with probability **d** (damping, usually 0.85)
# and jumps to a random page with probability **1 - d**. The PageRank of page *i* is the long-run
# fraction of time the surfer spends on *i*:
```

```
# 
# $$PR(i) = \frac{1-d}{N} + d \sum_{j \to i} \frac{PR(j)}{L(j)}$$
# 
# where *L(j)* is the number of outgoing links of *j* and *N* the number of pages.
# Pages with **no outgoing links** ("dangling nodes", e.g. a PDF or a leaf page) would leak
# probability, so their rank is spread evenly over all pages.
# 
# In matrix form, with the column-stochastic link matrix **M**, the Google matrix is
# $G = dM + \frac{1-d}{N}\mathbf{1}\mathbf{1}^T$ and PageRank is its principal eigenvector,
# found by **power iteration**: $r_{t+1} = G\, r_t$ until $\lVert r_{t+1} - r_t \rVert_1 < \varepsilon$.

# %% [markdown]
# ### 1.1 Tiny worked example (4 pages), same style as the reference Kaggle notebook

# %%
toy_links = {"A": ["B", "C"], "B": ["C"], "C": ["A"], "D": ["C"]}
pages = sorted(toy_links)
N = len(pages)
idx = {p: i for i, p in enumerate(pages)}

# column-stochastic transition matrix M[i, j] = 1/L(j) if j links to i
M = np.zeros((N, N))
for j, outs in toy_links.items():
    for i in outs:
        M[idx[i], idx[j]] = 1 / len(outs)
print("Transition matrix M:\n", pd.DataFrame(M, index=pages, columns=pages).round(3))

d = 0.85
G = d * M + (1 - d) / N * np.ones((N, N))
r = np.ones(N) / N
for it in range(100):
    r_new = G @ r
    if np.abs(r_new - r).sum() < 1e-10:
        break
    r = r_new
print(f"\nConverged after {it} iterations")
print(pd.Series(r_new, index=pages, name="PageRank").round(4))
print("\nNetworkX check:", {k: round(v, 4) for k, v in nx.pagerank(nx.DiGraph([(a, b) for a, bs in
toy_links.items() for b in bs]), alpha=d).items()})

# %% [markdown]
# Page **C** wins: three pages point to it. **A** is second because C (the most important page)
# gives all its rank to A. **D** has no incoming links, so it only gets the teleport share (1-d)/N.

# %% [markdown]
# ## 2. Our PageRank implementations
# 
# * `pagerank_simple`: a direct, readable translation of the formula using Python dicts.
#   Good for teaching and small graphs.
# * `pagerank_sparse`: the same algorithm with a SciPy sparse matrix. It is needed for the
#   28.9M-edge Wikipedia graph, where a dense N×N matrix would need ~80 GB.
# 
# Both handle dangling nodes and stop when the L1 change between iterations is below `tol`.

# %%
def pagerank_simple(graph, d=0.85, tol=1e-8, max_iter=200):
    """graph: dict node → list of out-neighbours (directed). Returns (ranks dict, history list)."""
    nodes = set(graph)
    for outs in graph.values():
        nodes.update(outs)
    nodes = sorted(nodes, key=str)
    N = len(nodes)
    out = {u: list(graph.get(u, [])) for u in nodes}
    incoming = {u: [] for u in nodes}
    for u, outs in out.items():
        for v in outs:
            incoming[v].append(u)
    dangling = [u for u in nodes if len(out[u]) == 0]

    pr = {u: 1 / N for u in nodes}
    history = []
    for _ in range(max_iter):
        dangling_mass = sum(pr[u] for u in dangling)
        new = {}
```

```
        for v in nodes:
            s = sum(pr[u] / len(out[u]) for u in incoming[v])
            new[v] = (1 - d) / N + d * (s + dangling_mass / N)
        delta = sum(abs(new[u] - pr[u]) for u in nodes)
        history.append(delta)
        pr = new
        if delta < tol:
            break
    return pr, history


def pagerank_sparse(src, dst, n, d=0.85, tol=1e-8, max_iter=200, weights=None):
    """Edges src[k] → dst[k] given as integer arrays with ids 0..n-1.
    Returns (rank vector, history of L1 deltas)."""
    src = np.asarray(src); dst = np.asarray(dst)
    w = np.ones(len(src)) if weights is None else np.asarray(weights, dtype=float)
    A = sp.csr_matrix((w, (src, dst)), shape=(n, n))          # A[i, j] = weight of i → j (duplicates
summed)
    out_w = np.asarray(A.sum(axis=1)).ravel()
    dangling = out_w == 0
    inv_out = np.zeros(n); inv_out[~dangling] = 1.0 / out_w[~dangling]
    P_T = (sp.diags(inv_out) @ A).T.tocsr()                   # P_T[j, i] = prob. of moving i → j
    r = np.full(n, 1.0 / n)
    history = []
    for _ in range(max_iter):
        r_new = d * (P_T @ r) + (d * r[dangling].sum() + (1 - d)) / n
        delta = np.abs(r_new - r).sum()
        history.append(delta)
        r = r_new
        if delta < tol:
            break
    return r / r.sum(), history


def compare_with_networkx(ours: dict, nx_pr: dict, k=10):
    """Agreement statistics between two PageRank dicts on the same node set."""
    keys = list(nx_pr)
    a = np.array([ours[u] for u in keys]); b = np.array([nx_pr[u] for u in keys])
    top_a = set(sorted(ours, key=ours.get, reverse=True)[:k])
    top_b = set(sorted(nx_pr, key=nx_pr.get, reverse=True)[:k])
    return {
        "max_abs_diff": float(np.max(np.abs(a - b))),
        "spearman": float(spearmanr(a, b).statistic),
        "kendall_tau": float(kendalltau(a, b).statistic),
        f"top{k}_overlap": len(top_a & top_b) / k,
    }


def top_table(pr: dict, k=10, label=None, extra=None):
    s = pd.Series(pr).sort_values(ascending=False).head(k)
    df = pd.DataFrame({"node": s.index, "pagerank": s.values})
    df.index = range(1, len(df) + 1); df.index.name = "rank"
    if label is not None:
        df.insert(1, "label", [label(u) for u in df["node"]])
    if extra:
        for name, fn in extra.items():
            df[name] = [fn(u) for u in df["node"]]
    return df


def plot_convergence(histories: dict, fname):
    plt.figure(figsize=(6, 3.5))
    for name, h in histories.items():
        plt.semilogy(range(1, len(h) + 1), h, marker="o", ms=3, label=name)
    plt.xlabel("iteration"); plt.ylabel("L1 change (log scale)")
    plt.title("Power-iteration convergence"); plt.legend()
    save_fig(fname)

# %% [markdown]
# ## 3. Dataset C: Zachary's Karate Club
#
# 34 members of a university karate club (Zachary, 1977). A conflict between the instructor
# (**node 1**, "Mr. Hi") and the administrator (**node 34**, "Officer") split the club in two.
# The edge list from networkrepository.com (`soc-karate.mtx`) is loaded if present in `data/`;
```

```
# otherwise we use NetworkX's built-in copy, which is the same 34-node, 78-edge network.
# We use the **1-based labels** of the original paper.

# %%
def load_karate():
    path = next(iter(glob.glob("data/**/soc-karate*.mtx", recursive=True) + glob.glob("soc-karate*.mtx")),
None)
    if path:
        edges, header_seen = [], False
        with open(path) as f:
            for line in f:
                if line.startswith("%") or not line.strip():
                    continue
                parts = line.split()
                if not header_seen and path.endswith(".mtx"):   # MatrixMarket size line "34 34 78"
                    header_seen = True
                    continue
                edges.append((int(parts[0]), int(parts[1])))
        G = nx.Graph(); G.add_edges_from(edges)
        source = f"networkrepository file {path}"
    else:
        G0 = nx.karate_club_graph()
        G = nx.relabel_nodes(G0, {u: u + 1 for u in G0})           # 1-based like the paper
        for u, v in G.edges():
            G[u][v].pop("weight", None)                               # use the unweighted version (as
soc-karate)
        source = "networkx.karate_club_graph() (same network as soc-karate)"
    faction = {u: d.get("club") for u, d in nx.karate_club_graph().nodes(data=True)}
    nx.set_node_attributes(G, {u + 1: c for u, c in faction.items()}, "club")
    return G, source

K, karate_source = load_karate()
print("Source:", karate_source)
print(f"Nodes: {K.number_of_nodes()}, edges: {K.number_of_edges()}, density: {nx.density(K):.3f}")

# %% [markdown]
# An undirected friendship counts as a link in **both** directions.

# %%
karate_graph = {u: list(K.neighbors(u)) for u in K.nodes()}
pr_k_simple, hist_k_simple = pagerank_simple(karate_graph)

nodes_k = sorted(K.nodes())
pos_of = {u: i for i, u in enumerate(nodes_k)}
src = [pos_of[u] for u, v in K.edges()] + [pos_of[v] for u, v in K.edges()]
dst = [pos_of[v] for u, v in K.edges()] + [pos_of[u] for u, v in K.edges()]
r, hist_k_sparse = pagerank_sparse(src, dst, len(nodes_k))
pr_k_sparse = {u: r[pos_of[u]] for u in nodes_k}

pr_k_nx = nx.pagerank(K, alpha=0.85, weight=None, tol=1e-10)

cmp_k = compare_with_networkx(pr_k_simple, pr_k_nx)
print(f"simple vs networkx: {cmp_k}")
print(f"sparse vs networkx: {compare_with_networkx(pr_k_sparse, pr_k_nx)}")
print(f"Iterations: simple={len(hist_k_simple)}, sparse={len(hist_k_sparse)}")

deg = dict(K.degree()); btw = nx.betweenness_centrality(K); eig = nx.eigenvector_centrality(K,
max_iter=1000)
role = {1: "Mr. Hi (instructor)", 34: "Officer (administrator)"}
top_k = top_table(pr_k_simple, 10, label=lambda u: role.get(u, ""),
                  extra={"club": lambda u: K.nodes[u]["club"], "degree": deg.get,
                         "betweenness": lambda u: round(btw[u], 3)})
top_k["pagerank"] = top_k["pagerank"].round(4)
top_k.to_csv(f"{OUT}/karate_top10.csv")
top_k

# %%
fig, ax = plt.subplots(figsize=(8, 6))
posK = nx.spring_layout(K, seed=7)
colors = ["#2a78d6" if K.nodes[u]["club"] == "Mr. Hi" else "#eb6834" for u in K.nodes()]
sizes = [pr_k_simple[u] * 9000 for u in K.nodes()]
nx.draw_networkx_edges(K, posK, alpha=0.3, ax=ax)
nx.draw_networkx_nodes(K, posK, node_size=sizes, node_color=colors, edgecolors="white", ax=ax)
nx.draw_networkx_labels(K, posK, font_size=8, font_color="white", ax=ax)
```

```
ax.scatter([], [], c="#2a78d6", label="Mr. Hi's faction"); ax.scatter([], [], c="#eb6834", label="Officer's faction")
ax.legend(loc="lower left", frameon=False); ax.set_title("Karate Club: node size = PageRank"); ax.axis("off")
save_fig("karate_network_pagerank.png")

# %%
fig, axes = plt.subplots(1, 2, figsize=(11, 4))
t = pd.Series(pr_k_simple).sort_values(ascending=False).head(10)
axes[0].barh([str(u) for u in t.index[::-1]], t.values[::-1],
             color=["#2a78d6" if K.nodes[u]["club"] == "Mr. Hi" else "#eb6834" for u in t.index[::-1]])
axes[0].set_xlabel("PageRank"); axes[0].set_ylabel("member"); axes[0].set_title("Top 10 members by PageRank")
axes[1].scatter([deg[u] for u in nodes_k], [pr_k_simple[u] for u in nodes_k], c="#2a78d6")
for u in [1, 34, 33, 3, 2]:
    axes[1].annotate(str(u), (deg[u], pr_k_simple[u]), xytext=(4, 2), textcoords="offset points")
axes[1].set_xlabel("degree (number of friends)"); axes[1].set_ylabel("PageRank"); axes[1].set_title("PageRank vs degree")
save_fig("karate_top10_and_degree.png")

# %% [markdown]
# ### 3.1 How PageRank compares with other centrality measures

# %%
cent = pd.DataFrame({"PageRank": pr_k_simple, "degree": nx.degree_centrality(K),
                     "betweenness": btw, "eigenvector": eig, "closeness": nx.closeness_centrality(K)})
rank_corr = cent.corr(method="spearman").round(3)
rank_corr.to_csv(f"{OUT}/karate_centrality_correlation.csv")
rank_corr

# %% [markdown]
# ### 3.2 Effect of the damping factor *d*

# %%
damps = [0.5, 0.65, 0.75, 0.85, 0.95]
sens = {}
iters = {}
for dval in damps:
    p, h = pagerank_simple(karate_graph, d=dval)
    sens[dval] = p; iters[dval] = len(h)
sens_df = pd.DataFrame(sens).loc[[34, 1, 33, 3, 2]]
sens_df.index = [f"node {u}" for u in sens_df.index]
plt.figure(figsize=(6, 3.5))
for name, row in sens_df.iterrows():
    plt.plot(damps, row.values, marker="o", label=name)
plt.xlabel("damping factor d"); plt.ylabel("PageRank"); plt.title("Karate: sensitivity to d"); plt.legend()
save_fig("karate_damping_sensitivity.png")
print("Iterations to converge per d:", iters)

# %%
plot_convergence({"dict version": hist_k_simple, "sparse version": hist_k_sparse}, "karate_convergence.png")
RESULTS["karate"] = {
    "source": karate_source, "nodes": K.number_of_nodes(), "edges": K.number_of_edges(),
    "iterations": len(hist_k_simple), "vs_networkx": cmp_k,
    "top10": [{"node": int(r_.node), "pagerank": float(r_.pagerank), "club": r_.club, "degree": int(r_.degree)} for r_ in top_k.itertuples()],
    "iterations_by_damping": iters,
}

# %% [markdown]
# **Findings (Karate).** Nodes **34** (Officer) and **1** (Mr. Hi) have by far the highest PageRank:
# the two leaders around whom the club split. PageRank correlates strongly with degree here
# because the graph is undirected (on undirected graphs PageRank is close to degree/2m), but it
# still reorders nodes with similar degree depending on how important their friends are.
# Our implementation matches NetworkX to within numerical precision.

# %% [markdown]
# ## 4. Dataset A: the VIT-AP website (`vitap.ac.in`)
#
# We build the web graph ourselves with a **polite breadth-first crawler**:
# * reads and obeys `robots.txt`
# * identifies itself with a User-Agent, waits `DELAY` seconds between requests
```

```
# * stays on `*.vitap.ac.in`, follows only HTML pages, stops after `MAX_PAGES`
# * also seeds from `sitemap.xml` if the site publishes one
#
# Nodes are pages, a directed edge *u → v* means page *u* contains a hyperlink to page *v*.
# The edge list is cached in `data/vitap_edges.csv`, so the crawl only has to run once
# (delete the file to re-crawl). Expect a few minutes for 300 pages.

# %%
START_URL   = "https://vitap.ac.in/"
ALLOWED     = "vitap.ac.in"          # host suffix to stay on
MAX_PAGES   = 300
DELAY       = 0.5                    # seconds between requests (be polite)
TIMEOUT     = 15
USER_AGENT  = "Mozilla/5.0 (compatible; ATSMA-student-PageRank-project/1.0; educational crawl)"
VITAP_CACHE = "data/vitap_edges.csv"
FORCE_HTTPS = True                   # treat http:// and https:// links as the same page
SKIP_EXT = re.compile(r"\.(pdf|jpe?g|png|gif|svg|webp|ico|mp4|webm|mp3|zip|rar|docx?|xlsx?|pptx?
|css|js|xml|json)$", re.I)

def normalize(url):
    url, _ = urldefrag(url)
    p = urlparse(url)
    if p.scheme not in ("http", "https"):
        return None
    host = p.netloc.lower()
    if host.startswith("www."):
        host = host[4:]
    path = re.sub(r"/+", "/", p.path or "/")
    if path ≠ "/" and path.endswith("/"):
        path = path[:-1]
    if path.endswith(("/index.html", "/index.php")):
        path = path.rsplit("/", 1)[0] or "/"
    q = "&".join(kv for kv in sorted(p.query.split("&")) if kv and not kv.lower().startswith(("utm_",
"fbclid", "gclid")))
    return f"{'https' if FORCE_HTTPS else p.scheme}://{host}{path}" + (f"?{q}" if q else "")

def in_scope(url):
    host = urlparse(url).netloc
    return (host == ALLOWED or host.endswith("." + ALLOWED)) and not SKIP_EXT.search(urlparse(url).path)

def crawl(start_url, max_pages=MAX_PAGES, delay=DELAY):
    import requests
    from bs4 import BeautifulSoup
    s = requests.Session(); s.headers["User-Agent"] = USER_AGENT
    rp = urllib.robotparser.RobotFileParser()
    seeds = [normalize(start_url)]
    try:
        txt = s.get(urljoin(start_url, "/robots.txt"), timeout=TIMEOUT).text
        rp.parse(txt.splitlines())
        sitemaps = [l.split(":", 1)[1].strip() for l in txt.splitlines() if
l.lower().startswith("sitemap:")]
    except Exception as e:
        print("robots.txt not reachable, crawling conservatively:", e)
        rp.parse([]); sitemaps = []
    for sm in sitemaps or [urljoin(start_url, "/sitemap.xml")]:
        try:
            xml = s.get(sm, timeout=TIMEOUT).text
            locs = re.findall(r"<loc>\s*([^<\s]+)\s*</loc>", xml)
            seeds += [u for u in (normalize(x) for x in locs) if u and in_scope(u)][:max_pages]
        except Exception:
            pass
    queue = collections.deque(dict.fromkeys(seeds)); seen = set(queue)
    edges, visited, titles, failed = [], [], {}, 0
    while queue and len(visited) < max_pages:
        url = queue.popleft()
        if not rp.can_fetch(USER_AGENT, url):
            continue
        try:
            resp = s.get(url, timeout=TIMEOUT, allow_redirects=True)
        except Exception:
            failed += 1; continue
        time.sleep(delay)
        if resp.status_code ≠ 200 or "html" not in resp.headers.get("Content-Type", ""):
            failed += 1; continue
```

```
        here = normalize(resp.url) or url
        visited.append(url)
        soup = BeautifulSoup(resp.text, "html.parser")
        titles[url] = (soup.title.get_text(strip=True) if soup.title else "")[:90]
        if here ≠ url:                                  # redirect: keep graph connected
            edges.append((url, here))
        for a in soup.find_all("a", href=True):
            v = normalize(urljoin(resp.url, a["href"].strip()))
            if v and in_scope(v) and v ≠ url:
                edges.append((url, v))
                if v not in seen:
                    seen.add(v); queue.append(v)
        if len(visited) % 25 == 0:
            print(f"  crawled {len(visited)} pages, {len(edges)} links, queue {len(queue)}")
    print(f"Done: {len(visited)} pages fetched, {failed} failed/non-HTML, {len(edges)} links found")
    return pd.DataFrame(edges, columns=["source", "target"]).drop_duplicates(), titles, visited

vitap_ok = False
os.makedirs("data", exist_ok=True)
if os.path.exists(VITAP_CACHE):
    vitap_edges = pd.read_csv(VITAP_CACHE)
    vitap_titles = json.load(open(VITAP_CACHE + ".titles.json")) if os.path.exists(VITAP_CACHE +
".titles.json") else {}
    vitap_visited = list(vitap_titles) or list(vitap_edges["source"].unique())
    print(f"Loaded cached crawl: {len(vitap_edges)} links")
    vitap_ok = len(vitap_edges) > 0
else:
    try:
        vitap_edges, vitap_titles, vitap_visited = crawl(START_URL)
        if len(vitap_edges):
            vitap_edges.to_csv(VITAP_CACHE, index=False)
            json.dump(vitap_titles, open(VITAP_CACHE + ".titles.json", "w"), indent=1)
            vitap_ok = True
        else:
            print("No links collected. The site may be blocking crawlers or render links with
JavaScript.")
    except Exception as e:
        print("Crawl failed (no internet?):", e)

# %% [markdown]
# **Graph construction choice.** Links that point to pages we did not fetch (because of the
# `MAX_PAGES` limit) are kept as nodes: they really receive links, they just have no known
# out-links, so PageRank treats them as dangling. We also report the graph restricted to the
# crawled pages only.

# %%
if vitap_ok:
    VG = nx.DiGraph(); VG.add_edges_from(vitap_edges.itertuples(index=False, name=None))
    print(f"VIT-AP web graph: {VG.number_of_nodes()} pages, {VG.number_of_edges()} hyperlinks")
    print(f"  crawled pages: {len(vitap_visited)}, dangling (no out-links known): {sum(1 for u in VG if
VG.out_degree(u) == 0)}")
    print(f"  weakly connected components: {nx.number_weakly_connected_components(VG)}, "
          f"largest SCC: {len(max(nx.strongly_connected_components(VG), key=len))} pages")

    vgraph = {u: list(VG.successors(u)) for u in VG}
    pr_v, hist_v = pagerank_simple(vgraph)
    pr_v_nx = nx.pagerank(VG, alpha=0.85, tol=1e-10)
    cmp_v = compare_with_networkx(pr_v, pr_v_nx)
    print("ours vs networkx:", cmp_v, "| iterations:", len(hist_v))

    hubs, auth = nx.hits(VG, max_iter=1000)
    short = lambda u: (urlparse(u).netloc.replace("vitap.ac.in", "").rstrip(".") + " " + (urlparse(u).path
or "/")).strip()
    top_v = top_table(pr_v, 15, label=lambda u: vitap_titles.get(u, ""),
                      extra={"in_links": VG.in_degree, "out_links": VG.out_degree})
    top_v["pagerank"] = top_v["pagerank"].round(4)
    top_v.to_csv(f"{OUT}/vitap_top15.csv")
    display(top_v)
else:
    print("Skipping VIT-AP analysis: no crawl data. Run this notebook with internet access.")

# %%
if vitap_ok:
    t = pd.Series(pr_v).sort_values(ascending=False).head(15)[::-1]
```

```
    plt.figure(figsize=(9, 5.5))
    plt.barh([short(u)[:55] for u in t.index], t.values, color="#2a78d6")
    plt.xlabel("PageRank"); plt.title("VIT-AP website: top 15 pages by PageRank")
    save_fig("vitap_top15.png")

    # in-links vs PageRank and PageRank vs HITS authority
    fig, axes = plt.subplots(1, 2, figsize=(11, 4))
    nodes_v = list(VG)
    axes[0].scatter([VG.in_degree(u) for u in nodes_v], [pr_v[u] for u in nodes_v], s=10, alpha=0.6,
c="#2a78d6")
    axes[0].set_xlabel("in-links"); axes[0].set_ylabel("PageRank"); axes[0].set_title("PageRank vs number
of in-links")
    axes[1].scatter([auth[u] for u in nodes_v], [pr_v[u] for u in nodes_v], s=10, alpha=0.6, c="#eb6834")
    axes[1].set_xlabel("HITS authority score"); axes[1].set_ylabel("PageRank");
axes[1].set_title("PageRank vs HITS authority")
    save_fig("vitap_pagerank_vs_inlinks.png")

# %%
if vitap_ok:
    top40 = [u for u, _ in sorted(pr_v.items(), key=lambda x: -x[1])[:40]]
    H = VG.subgraph(top40)
    plt.figure(figsize=(10, 8))
    posV = nx.spring_layout(H, seed=3, k=0.9)
    nx.draw_networkx_edges(H, posV, alpha=0.15, arrows=True, arrowsize=6)
    nx.draw_networkx_nodes(H, posV, node_size=[pr_v[u] * 25000 for u in H], node_color="#2a78d6",
alpha=0.85)
    lbl = {u: short(u)[:28] for u in top40[:15]}
    nx.draw_networkx_labels(H, posV, labels=lbl, font_size=7)
    plt.title("VIT-AP: link structure among the 40 highest-ranked pages (size = PageRank)");
plt.axis("off")
    save_fig("vitap_top40_graph.png")
    plot_convergence({"VIT-AP": hist_v}, "vitap_convergence.png")
    RESULTS["vitap"] = {
        "pages_crawled": len(vitap_visited), "nodes": VG.number_of_nodes(), "edges": VG.number_of_edges(),
        "iterations": len(hist_v), "vs_networkx": cmp_v,
        "top15": [{"url": r_.node, "title": r_.label, "pagerank": float(r_.pagerank), "in_links":
int(r_.in_links)} for r_ in top_v.itertuples()],
    }

# %% [markdown]
# **What to look for (VIT-AP).** Pages linked from the global header/footer menu (home page,
# admissions, academics, contact, placements) appear on every page, so they collect links from
# the whole site and dominate PageRank. This is the "navigation effect" of web analytics: rank on a
# single site mostly reflects **site architecture**, not content popularity. Deep content pages
# (news items, department pages) sit in the long tail.

# %% [markdown]
# ## 5. Dataset B: Wikipedia Link Graph (100K pages, ~28.9M links)
#
# **Getting the data** (one of):
# 1. Automatic: `kagglehub` downloads it (works on Kaggle/Colab, and locally once you have a Kaggle
account;
#    if asked for credentials, put your `kaggle.json` API token in `~/.kaggle/`).
# 2. Manual: download the zip from https://www.kaggle.com/datasets/kutayahin/wikipedia-link-graph-100k,
#    unzip it into `data/wikipedia/`.
#
# The loader below **inspects the files and detects the edge list automatically**
(CSV/TSV/TXT/Parquet/JSON),
# then reads the edges in chunks and maps page names to integer ids, so ~29M edges fit in a few
# hundred MB of RAM. If your laptop struggles, set `MAX_EDGES` (e.g. 5_000_000) or run on Kaggle/Colab.

# %%
WIKI_DIR  = "data/wikipedia"
MAX_EDGES = None          # e.g. 5_000_000 to use only the first 5M links on a small laptop

def get_wiki_dir():
    for z in glob.glob(os.path.join(WIKI_DIR, "*.zip")) + glob.glob("*wikipedia*link*graph*.zip"):
        import zipfile
        print("Unzipping", z); zipfile.ZipFile(z).extractall(WIKI_DIR); os.remove(z) if
z.startswith(WIKI_DIR) else None
    if glob.glob(os.path.join(WIKI_DIR, "**", "*.*"), recursive=True):
        return WIKI_DIR
    for p in glob.glob("/kaggle/input/*wiki*"):                    # running on Kaggle with the dataset
attached
```

```
            return p
        try:
            import kagglehub
            return kagglehub.dataset_download("kutayahin/wikipedia-link-graph-100k")
        except Exception as e:
            print("Could not download automatically:", str(e)[:200])
            return None

    wiki_dir = get_wiki_dir()
    wiki_files = []
    if wiki_dir:
        for f in sorted(glob.glob(os.path.join(wiki_dir, "**", "*"), recursive=True)):
            if os.path.isfile(f):
                wiki_files.append(f)
                print(f"{os.path.getsize(f)/1e6:10.1f} MB  {os.path.relpath(f, wiki_dir)}")

    # %%
    TAB_EXT = (".csv", ".tsv", ".txt", ".csv.gz", ".tsv.gz", ".txt.gz", ".parquet", ".json", ".jsonl",
".jsonl.gz", ".json.gz")
    SRC_PAT = re.compile(r"^(src|source|from|page|page_?id|page_?title|source_?
(id|title|page)|origin|article|node_?1|u)$", re.I)
    DST_PAT = re.compile(r"^(dst|dest|destination|target|to|link|link_?(id|title)|target_?
(id|title|page)|outlink|node_?2|v)$", re.I)

    def detect_sep(path):
        import gzip, csv
        opener = gzip.open if path.endswith(".gz") else open
        with opener(path, "rt", errors="ignore") as f:
            line = f.readline()
        if "\t" in line: return "\t"
        try:
            return csv.Sniffer().sniff(line, delimiters=",;| ").delimiter
        except csv.Error:
            return ","

    def sniff(path, n=5):
        if path.endswith(".parquet"):
            return pd.read_parquet(path).head(n)
        if ".json" in path:
            return pd.read_json(path, lines=".jsonl" in path, nrows=n) if ".jsonl" in path else None
        return pd.read_csv(path, sep=detect_sep(path), nrows=n)

    def pick_columns(cols):
        s = next((c for c in cols if SRC_PAT.match(str(c).strip())), None)
        t = next((c for c in cols if DST_PAT.match(str(c).strip()) and c != s), None)
        if s is None or t is None:
            s, t = cols[0], cols[1]
        return s, t

    def load_wiki_edges(files, max_edges=None):
        tab = [f for f in files if f.lower().endswith(TAB_EXT)]
        if not tab:
            raise FileNotFoundError("No tabular files found in the Wikipedia folder")
        for f in tab:
            try:
                print(f"--- {os.path.basename(f)}"); print(sniff(f))
            except Exception as e:
                print("   (could not preview:", e, ")")
        edge_file = max(tab, key=os.path.getsize)                  # the link list is by far the biggest file
        print("\nUsing as edge list:", edge_file)
        node_files = [f for f in tab if f != edge_file]

        # JSON adjacency format: one record per page with a list of links
        if ".json" in edge_file:
            df = pd.read_json(edge_file, lines=".jsonl" in edge_file)
            tcol = next(c for c in df.columns if df[c].map(lambda x: isinstance(x, list)).any())
            scol = next(c for c in df.columns if c != tcol)
            df = df[[scol, tcol]].explode(tcol).dropna()
            df.columns = ["source", "target"]
            chunks = [df]
        elif edge_file.endswith(".parquet"):
            df = pd.read_parquet(edge_file); s, t = pick_columns(list(df.columns))
            chunks = [df[[s, t]].rename(columns={s: "source", t: "target"})]
        else:
```

```
        sep = detect_sep(edge_file)
        head = sniff(edge_file); s, t = pick_columns(list(head.columns))
        print(f"source column = {s!r}, target column = {t!r}")
        chunks = (c.rename(columns={s: "source", t: "target"})
                  for c in pd.read_csv(edge_file, sep=sep, usecols=[s, t], chunksize=2_000_000, dtype=str,
on_bad_lines="skip"))

    ids, names = {}, []
    src_parts, dst_parts, total = [], [], 0
    for c in chunks:
        c = c.dropna()
        if max_edges is not None and total + len(c) > max_edges:
            c = c.iloc[: max_edges - total]
        for col in ("source", "target"):
            for v in pd.unique(c[col].astype(str).values):
                if v not in ids:
                    ids[v] = len(names); names.append(v)
        src_parts.append(c["source"].astype(str).map(ids).to_numpy(np.int32))
        dst_parts.append(c["target"].astype(str).map(ids).to_numpy(np.int32))
        total += len(c)
        print(f"  read {total:,} links, {len(names):,} distinct pages")
        if max_edges is not None and total ⩾ max_edges:
            break
    src = np.concatenate(src_parts); dst = np.concatenate(dst_parts)

    # optional id → title lookup from a smaller "nodes/pages" file
    title_of = {}
    for f in node_files:
        try:
            nd = pd.read_parquet(f) if f.endswith(".parquet") else pd.read_csv(f, sep=detect_sep(f),
dtype=str)
            idc = next((c for c in nd.columns if re.search(r"id", str(c), re.I)), None)
            tc = next((c for c in nd.columns if re.search(r"title|name|page", str(c), re.I) and c ≠ idc),
None)
            if idc and tc:
                title_of = dict(zip(nd[idc].astype(str), nd[tc].astype(str)))
                print(f"Titles from {os.path.basename(f)}: {len(title_of):,}")
                break
        except Exception:
            pass
    return src, dst, names, title_of

wiki_ok = False
if wiki_files:
    t0 = time.time()
    w_src, w_dst, w_names, w_title = load_wiki_edges(wiki_files, MAX_EDGES)
    print(f"Loaded in {time.time()-t0:.0f}s")
    wiki_ok = True
else:
    print("Wikipedia data not found: download it into data/wikipedia/ (see instructions above) and re-
run.")

# %% [markdown]
# ### 5.1 Cleaning
# * remove self-links and duplicate links
# * The dataset has ~100K pages but links may point to articles **outside** the 100K set.
#   We keep the graph **closed** (only links whose target is also one of the source pages) when the
#   number of targets is much larger than the number of sources; otherwise PageRank would be dominated
#   by millions of pages we know nothing about. Set `CLOSE_GRAPH = False` to keep everything.

# %%
CLOSE_GRAPH = "auto"
if wiki_ok:
    keep = w_src ≠ w_dst
    pairs = np.unique((w_src[keep].astype(np.int64) << 32) | w_dst[keep].astype(np.int64))
    s_, d_ = (pairs >> 32).astype(np.int32), (pairs & 0xFFFFFFFF).astype(np.int32)
    n_sources = len(np.unique(s_)); n_all = len(w_names)
    print(f"after removing self/duplicate links: {len(s_):,} links; pages with out-links: {n_sources:,};
all pages mentioned: {n_all:,}")
    close = (n_all > 1.5 * n_sources) if CLOSE_GRAPH == "auto" else CLOSE_GRAPH
    if close:
        is_src = np.zeros(n_all, bool); is_src[s_] = True
        m = is_src[d_]; s_, d_ = s_[m], d_[m]
        print(f"closed graph: kept {m.mean():.1%} of links")
```

```
    # re-index to 0..n-1
    used = np.unique(np.concatenate([s_, d_]))
    remap = np.full(n_all, -1, np.int64); remap[used] = np.arange(len(used))
    ws, wd = remap[s_], remap[d_]
    wn = len(used)
    wiki_label = lambda i: w_title.get(w_names[used[i]], w_names[used[i]])
    out_deg = np.bincount(ws, minlength=wn); in_deg = np.bincount(wd, minlength=wn)
    print(f"Final Wikipedia graph: {wn:,} pages, {len(ws):,} links, "
          f"mean out-degree {len(ws)/wn:.1f}, dangling pages {(out_deg==0).sum():,}")

# %%
if wiki_ok:
    t0 = time.time()
    r_w, hist_w = pagerank_sparse(ws, wd, wn, d=0.85, tol=1e-8)
    print(f"PageRank on {wn:,} nodes / {len(ws):,} edges: {len(hist_w)} iterations, {time.time()-t0:.1f}s")
    order = np.argsort(-r_w)
    top_w = pd.DataFrame({"page": [wiki_label(i) for i in order[:20]], "pagerank": r_w[order[:20]].round(6),
                          "in_links": in_deg[order[:20]], "out_links": out_deg[order[:20]]},
                         index=pd.RangeIndex(1, 21, name="rank"))
    top_w.to_csv(f"{OUT}/wikipedia_top20.csv")
    top_by_in = pd.Series(in_deg).sort_values(ascending=False).head(20)
    print("Top-20 overlap with simply counting in-links:", len(set(order[:20]) & set(top_by_in.index)), "/20")
    display(top_w)

# %% [markdown]
# ### 5.2 Verifying against NetworkX
# Building a NetworkX graph with ~29M edges needs tens of GB of RAM, so we check correctness on a
# **random induced subgraph** of 20,000 pages: our sparse implementation and `nx.pagerank` must agree there.

# %%
if wiki_ok:
    SUB = min(20_000, wn)
    pick = np.random.default_rng(0).choice(wn, SUB, replace=False)
    insub = np.zeros(wn, bool); insub[pick] = True
    m = insub[ws] & insub[wd]
    loc = np.full(wn, -1); loc[pick] = np.arange(SUB)
    ss, dd = loc[ws[m]], loc[wd[m]]
    r_sub, _ = pagerank_sparse(ss, dd, SUB)
    Gsub = nx.DiGraph(); Gsub.add_nodes_from(range(SUB)); Gsub.add_edges_from(zip(ss.tolist(), dd.tolist()))
    t0 = time.time(); pr_sub_nx = nx.pagerank(Gsub, alpha=0.85, tol=1e-10); t_nx = time.time() - t0
    cmp_w = compare_with_networkx({i: r_sub[i] for i in range(SUB)}, pr_sub_nx)
    print(f"subgraph: {SUB:,} nodes, {m.sum():,} edges; networkx took {t_nx:.1f}s")
    print("ours vs networkx:", cmp_w)

# %%
if wiki_ok:
    fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))
    t = top_w.head(15)[::-1]
    axes[0].barh([str(p)[:40] for p in t["page"]], t["pagerank"], color="#2a78d6")
    axes[0].set_xlabel("PageRank"); axes[0].set_title("Wikipedia 100K: top 15 articles")
    vals = np.sort(r_w)[::-1]
    axes[1].loglog(np.arange(1, len(vals) + 1), vals, color="#eb6834")
    axes[1].set_xlabel("rank (log)"); axes[1].set_ylabel("PageRank (log)"); axes[1].set_title("Rank-score distribution: a heavy tail")
    save_fig("wikipedia_top15_and_distribution.png")

    plt.figure(figsize=(6, 4))
    samp = np.random.default_rng(1).choice(wn, min(wn, 30000), replace=False)
    plt.loglog(in_deg[samp] + 1, r_w[samp], ".", ms=2, alpha=0.3, color="#2a78d6")
    plt.xlabel("in-links + 1 (log)"); plt.ylabel("PageRank (log)"); plt.title("Wikipedia: PageRank vs in-links")
    save_fig("wikipedia_pagerank_vs_inlinks.png")
    plot_convergence({"Wikipedia 100K": hist_w}, "wikipedia_convergence.png")

    share_top1 = float(np.sort(r_w)[::-1][: max(1, wn // 100)].sum())
    print(f"The top 1% of pages hold {share_top1:.1%} of all PageRank")
    RESULTS["wikipedia"] = {
        "nodes": int(wn), "edges": int(len(ws)), "closed_graph": bool(close), "iterations": len(hist_w),
        "dangling": int((out_deg == 0).sum()), "top1pct_share": share_top1, "vs_networkx_subgraph": cmp_w,
```

```
 760          "top20": [{"page": str(r_.page), "pagerank": float(r_.pagerank), "in_links": int(r_.in_links)} for
r_ in top_w.itertuples()],
 761      }
 762
 763  # %% [markdown]
 764  # ## 6. Comparison across the three networks
 765
 766  # %%
 767  rows = []
 768  if "karate" in RESULTS:
 769      rows.append(["Karate Club", "undirected social", RESULTS["karate"]["nodes"], RESULTS["karate"]
["edges"], RESULTS["karate"]["iterations"], RESULTS["karate"]["vs_networkx"]["max_abs_diff"]])
 770  if "vitap" in RESULTS:
 771      rows.append(["VIT-AP website", "directed web (crawl)", RESULTS["vitap"]["nodes"], RESULTS["vitap"]
["edges"], RESULTS["vitap"]["iterations"], RESULTS["vitap"]["vs_networkx"]["max_abs_diff"]])
 772  if "wikipedia" in RESULTS:
 773      rows.append(["Wikipedia 100K", "directed encyclopedia", RESULTS["wikipedia"]["nodes"],
RESULTS["wikipedia"]["edges"], RESULTS["wikipedia"]["iterations"], RESULTS["wikipedia"]["vs_networkx_subgraph"]
["max_abs_diff"]])
 774  summary = pd.DataFrame(rows, columns=["network", "type", "nodes", "edges", "iterations (tol 1e-8)", "max
|ours - networkx|"])
 775  summary.to_csv(f"{OUT}/summary.csv", index=False)
 776  json.dump(RESULTS, open(f"{OUT}/results.json", "w"), indent=2, default=str)
 777  summary
 778
 779  # %% [markdown]
 780  # ## Conclusions
 781  # 1. **Correctness.** Our from-scratch power iteration reproduces NetworkX's PageRank on all three
 782  #    networks (differences around 1e-9) and converges in a few dozen iterations, as theory predicts
 783  #    (the error shrinks by roughly a factor *d* = 0.85 each step).
 784  # 2. **Karate Club.** PageRank identifies the two leaders (34 and 1) who split the club. On an undirected
 785  #    graph it behaves much like degree centrality, yet it still rewards being connected to important
people.
 786  # 3. **VIT-AP website.** Ranking is driven by site navigation: pages in the global menu/footer collect
 787  #    the most rank. This shows how internal linking (site architecture) shapes what a search engine
 788  #    considers important, which is useful for SEO and information-architecture audits.
 789  # 4. **Wikipedia.** At ~29M links PageRank needs sparse matrices but still finishes in seconds to minutes.
 790  #    The scores are extremely skewed (a heavy tail): a small set of general articles (countries, years,
 791  #    major topics) concentrate most of the rank, and PageRank differs from simply counting in-links.
 792  # 5. **Damping factor.** Higher *d* follows links longer and makes ranks more unequal but slows
convergence;
 793  #    lower *d* pulls everything toward uniform.
 794  #
 795  # **Limitations.** The crawl is capped at `MAX_PAGES` and ignores JavaScript-rendered links; the Wikipedia
 796  # graph is a 100K-page sample, so pages outside it are excluded; Karate is tiny and undirected.
 797  #
 798  # **References.** Brin & Page (1998), *The Anatomy of a Large-Scale Hypertextual Web Search Engine*;
 799  # Zachary (1977), *An Information Flow Model for Conflict and Fission in Small Groups*;
 800  # Rossi & Ahmed, *The Network Data Repository* (networkrepository.com); Kaggle: kutayahin/wikipedia-link-
graph-100k;
 801  # Kaggle reference notebook: vinitkp/page-rank-algorithm.
```

Appendix B Demo application core and requirements

B.1 PageRank in JavaScript (demo-app/index.html)

The browser port of `pagerank_simple()` used by every view of the demo app.

```javascript
function pagerank(nodes, out, d = 0.85, tol = 1e-8, maxIter = 200, keep = false) {
  const N = nodes.length, idx = new Map(nodes.map((u, i) => [u, i]));
  const inc = nodes.map(() => []);
  const L = nodes.map(u => (out.get(u) || []).length);
  nodes.forEach((u, i) => (out.get(u) || []).forEach(v => inc[idx.get(v)].push(i)));
  const dangling = L.map((l, i) => l === 0 ? i : -1).filter(i => i >= 0);
  let pr = new Float64Array(N).fill(1 / N);
  const hist = [], snaps = keep ? [Array.from(pr)] : null;
  for (let t = 0; t < maxIter; t++) {
    let dm = 0; for (const i of dangling) dm += pr[i];
    const nw = new Float64Array(N);
    let delta = 0;
    for (let v = 0; v < N; v++) {
      let s = 0; for (const u of inc[v]) s += pr[u] / L[u];
      nw[v] = (1 - d) / N + d * (s + dm / N);
      delta += Math.abs(nw[v] - pr[v]);
    }
    pr = nw; hist.push(delta); if (keep) snaps.push(Array.from(pr));
    if (delta < tol) break;
  }
  return { pr, hist, snaps, dangling: dangling.length };
}
```

B.2 requirements.txt

```
networkx>=3.0
numpy
scipy
pandas
matplotlib
requests
beautifulsoup4
kagglehub
jupyter
```